Meeting Course Outcomes

Sabrina Ozburn

CS-499 Computer Science Capstone

Prof. Wasim Alim

8/14/2026

Outcome 1: Collaborative Environments and Organizational Decision-Making

- **Alignment Matrix:** This program outcome is met by transforming raw datasets into highly interpretable, reactive visual assets that empower non-technical organizational leadership to execute data-driven choices. By engineering an interactive dashboard that maps specialized animal rescue categories (such as water rescue or disaster-tracking dogs) to distinct geographic points and demographic breakdowns, the software directly supports strategic operational scheduling. Furthermore, my academic background involved working across simulated sprint intervals and Scrum frameworks, requiring me to translate high-level organizational requirements into discrete technical tasks. This dual experience enables me to build highly collaborative development environments that remain responsive to shifting organizational metrics, mirroring the strict configuration control guidelines I followed in the aerospace industry.

Outcome 2: Professional Oral, Written, and Visual Communications

- **Alignment Matrix:** This outcome is satisfied through the multi-tier documentation, structured markdown configuration setups, and video-recorded technical walkthroughs produced across the portfolio lifecycle. Software engineering requires technical soundness to match clear human delivery. My engineering narratives and code comments skip abstract jargon, explicitly articulating complex architectural concepts—such as recursive partitioning or server-side constraints—in universal language. By adapting my design explanations to meet the needs of academic evaluators, code reviewers, and business stakeholders simultaneously, I demonstrate complete mastery of professional presentation standards. This visual and written articulation proves that I can confidently

market enterprise engineering solutions and defend structural design choices in high-visibility settings.

Outcome 3: Computing Solutions and Algorithmic Design Choice Trade-offs

- **Alignment Matrix:** This outcome is satisfied through the intentional design and deployment of the custom memory optimization layer built into the application controller. In Category Two, I explicitly intercepted native web components and replaced generic framework functions with a manual, recursive QuickSort routine utilizing a divide-and-conquer processing loop. Implementing this structure forced an active evaluation of computational trade-offs, where I balanced local memory usage against external database server processing cycles. By structuring datasets as addressable sequential lists of key-value dictionaries, my algorithm maps array transformations to a highly optimized average time complexity of $O(\log n)$, demonstrating a sophisticated ability to maximize data-manipulation speed under strict hardware constraints.

Outcome 4: Well-Founded and Innovative Techniques, Skills, and Tools

- **Alignment Matrix:** This outcome is fully met by transitioning the application away from unmanaged script environments and engineering a production-grade standalone web application within Visual Studio Code. By utilizing the Python Dash and Plotly frameworks, isolating software packages within virtual environment sandboxes, and fully decoupling my system architecture into a strict modular Repository pattern, I demonstrate proficiency in modern computing practices. This migration from interactive notebooks to native execution run loops ensures full environmental portability, demonstrating my

ability to deliver scalable, standalone software assets that integrate seamlessly into industry-standard deployment pipelines.

Outcome 5: Security Mindset, Adversarial Exploits, and Data Resource Privacy

- **Alignment Matrix:** This critical software safety outcome is met by implementing strict data governance parameters at both the data-access and storage layers of the full-stack system. At the data access tier, I constructed defensive error traps and data-emptiness boundaries that gracefully intercept missing or malformed records before runtime display components can crash. More importantly, at the backend database layer, I implemented strict server-side validation schema routines that automatically test all input arguments for structural validity, type matching, and completeness before executing write mutations. Combined with the removal of hardcoded configuration tokens and their replacement with localized .env environments, this defensive programming prevents adversarial exploits, data corruption, or script injections, mirroring the safety-critical precision metrics of the aerospace sector.